6a. Font consistency

What it detects: text set in a different font family from the text around it, the trace a value leaves when it is typed in after the document was produced (an amount retyped in Times on a Helvetica invoice, or pasted in Arial on a Calibri letter before it was printed and scanned).

Code: `services/forensics/font_consistency.py` (`analyze_font_consistency`), run inside `tasks/document_processing.py` (`process_document`) right after OCR and **before the document is marked complete**, so a case is never scored without it. Stores `check_type = font_consistency` .

It runs in the extraction task, not as its own task, because scanned pages need that task's OCR font estimates. A failure is stored as a `failed` check and never fails the extraction.

Two sources, chosen per page

Page	Source	How the font is known	Finding severity
Digitally produced (has a text layer)	`text_layer`	The font **name** of every text run, read with PyMuPDF. Exact	`high`
Scanned (no text layer)	`ocr`	Azure Document Intelligence Layout's **styleFont** add-on: an estimated similar font family per recognized word (`services/ocr_service.py`)	`medium` when 5+ words share the odd font, else `low` (shown, not scored)
Neither (scan with the add-on off)	`none`	Not analysed. Fonts there are left to the visual review	

The text layer is exact, so one run in an out-of-place font is reported and scored, at any size. OCR font recognition is an estimate and noisy. It labels one body font "Segoe UI" on some words and "Tahoma" on others, and guesses wildly on small print and signatures. So on scans:

- families are compared as broad **visual classes** (humanist sans, grotesque sans, geometric sans, serif, slab serif, monospace). Unknown families are not used;
- only labels with confidence **≥ 0.95** on **body-size** text (0.8–1.3 × the page's median word height) that is not italic or handwritten are used;
- **every** word in an out-of-place class is reported and boxed, even a single one. It is **scored** (`medium`) only once that class covers **at least 5 words** of the page; below that it is `low` : shown for the reviewer, not scored.

Why the split: on scans of genuine documents, OCR font recognition mislabels a few single words per page (phone numbers, single digits, "Total", "DUE:"), never 5 in one class, while scans with retyped amounts reached 6–8. Scoring single words would mark genuine scans risky; showing them lets the reviewer judge a one-value edit by eye. On a digital PDF every mismatch is exact and always scored.

The rule

A run is flagged when its family is a **minor** font on the page (under 25 % of the page's characters in that script) and it sits among text of similar size (within 15 % on a text layer, 50 % on a scan) in a more common family:

- **row:** on the same visual row (`Subtotal:` in Helvetica, `18,450.00 USD` in Times);
- **column:** in a left- or right-aligned column of 3+ runs where most runs are in more common families (a whole line item retyped in Courier between line items in DejaVu Sans).

Not flagged, by design:

- weight/style (Bold, Italic…), subset tags, and metric-compatible substitutes (Arial = Helvetica, Times New Roman = Times, Courier New = Courier);
- symbol/icon fonts and script fonts used for typed signatures;
- text of a different size (a letterhead or heading in a display font);
- text in another writing system: runs are compared only within one script, so Arabic set in an Arabic font beside Latin text in Helvetica is normal;
- a family that sets 25 % or more of the page's text, or most of a column of 4+ runs: that is the document's design.

Output shape (real, truncated)


```
{
  "result": "flag",
  "details": [
    {"finding": "font_family_inventory", "severity": "info",
     "description": "1 of 1 page(s) analysed (1 scanned, using OCR font recognition); 3 font families: …",
     "data": {"families": {"humanist sans": 905, "grotesque sans": 41, "…": 0},
              "pages": [{"page": 1, "source": "ocr"}]}},
    {"finding": "font_inconsistency", "severity": "medium",
     "description": "Page 1 (scanned): '22,000' looks like a different font (grotesque sans) from the text
around it (humanist sans), per OCR font recognition — …",
     "page": 1, "bounding_box": {"page": 1, "x": 0.3851, "y": 0.4129, "width": 0.0627, "height": 0.0181},
     "data": {"text": "22,000", "font": "Arial, Helvetica, sans-serif", "family": "grotesque sans",
              "expected_family": "humanist sans", "font_size": 10.8, "source": "ocr"}}
  ]
}
```


Each `font_inconsistency` carries `page` and a normalized `bounding_box`. The Case Detail viewer draws it as a solid fuchsia **Font mismatch** box, and so does the exported case report.

Known limitations

- **PDF only.**
- **Scans:** needs the styleFont add-on (`AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT`, on by default, billed by Azure per page) and catches only edits spanning several words (see above). Heading-size and small-print text on scans is not compared.
- **Digital PDFs:** an edit pasted in as an image is invisible to it (ELA, copy-move and the visual review cover those). A forger who retypes values in the document's own font, or a design that deliberately mixes families in a small area, defeats it.

Scans converted to editable text

Acrobat "Edit scanned document" (and similar converters) re-create every word of a scan as live text in subset fonts they generate per guessed style, named `<Family>-NNNN` (`Comic Sans MS-Bold-6003`, `Arial-Bold-6000`). The `-NNNN` number is not part of the family. On a page with at least 60 % of its characters in such subsets, the family comparison is **not** used — the converter's guesses differ word to word on small print, so two subsets side by side (`03` in `Comic Sans MS-BoldItalic-6002` beside `14` in `Arial-Bold-6000`) are not evidence. Instead, a run in a **full** font (no `-NNNN`) on a row of the converter's subsets of similar size is flagged `high`: it was typed in after the conversion. The finding lists the digits the neighbouring subset holds (read from its ToUnicode map) — a subset only contains the glyphs originally printed with it, so `digits_not_in_original` hints at what the value was.

Size changes inside a number

On every text-layer page, a number whose characters are set ≥ 0.2 pt apart (`36,000` with `6` and `000` at 11.7 pt inside a 12 pt figure) is noted on the font finding covering it (`data.size_change`), or reported on its own as `font_size_inconsistency` (`high`). Ordinals like `1st` are not numbers.

A font embedded more than once

A document exported in one pass embeds each font and style once, as one subset tagged `ABCDEF+` . An edit session that types text in adds **another subset of the same font** (PyMuPDF reports the tags only when asked: `TOOLS.set_subset_fontnames(True)` , set for the read and restored). Fonts are grouped by family and style ignoring the tag; the subset holding the most text is the main copy. Text in an extra copy is flagged `font_subset_split` (`high`) when:

- it is on the **same line** as text in the main copy ("AL REEM" beside "SALEH MOHAMED ALHARTHI"), or
- it is an **amount in a column** whose other amounts are in the main copy ("1,450.00" above "4,575.00"), or
- it is an **amount in a copy holding only digits and punctuation** (`, . 0 1 2 3 4 5 6`).

An extra copy by itself is not enough: Word puts a stray en-dash, or a second bold-italic heading on another line, in a subset of its own. `data` carries `subset` , `main_subset` , `subset_glyphs` , `digits_only_subset` , `same_row_as` , `same_column_as` . CASE-39CB18BF: 17 spans — the amounts of both columns in a digits-only Times New Roman copy, the totals and amount in words in a third bold copy, the student's first name, ID and class in a second bold copy.

Risk rules fed

`rule_id`	Finding	Fires on	Weight	Severity
`font.inconsistency` (v3)	`font_inconsistency` , `font_size_inconsistency` , `font_subset_split`	high (text layer)	40	high
`font.inconsistency_scanned`	`font_inconsistency`	medium (OCR, scans, 5+ words)	25	medium

`low` findings (a scan's odd font on fewer than 5 words) are not scored by any rule.